Clemens Höglinger

Zeitlich robuste Erkennung und Zustandsmodellierung von Katzen und Beutetieren mittels Deep Learning

Bachelorarbeit

zur Erlangung des akademischen Grades

Bachelor of Science in Engineering (BSc)

Bachelorstudiengang
Software Engineering
Fachhochschule Oberösterreich
Hagenberg

Jahr der Einreichung: 2026

Betreuung: Dr. Gerald Adam Zwettler

© Copyright 2026 Clemens Höglinger

Diese Arbeit wird unter den Bedingungen der Creative Commons Lizenz *Attribution-NonCommercial-NoDerivatives 4.0 International* (CC BY-NC-ND 4.0) veröffentlicht – siehe https://creativecommons.org/licenses/by-nc-nd/4.0/.

Eidesstattliche Erklärung

Ich erkläre eidesstattlich, dass ich die vorliegende Arbeit selbständig verfasst und nur die angegebenen Quellen und Hilfsmittel verwendet habe. Alle Stellen, die wörtlich oder inhaltlich den angegebenen Quellen entnommen wurden, sind als solche kenntlich gemacht. Die Arbeit wurde bisher in gleicher oder ähnlicher Form keiner anderen Prüfungsbehörde vorgelegt und ist mit dem elektronisch eingereichten Textdokument identisch.

Hagenberg, am 20. August 2026

Clemens Höglinger

Dokumentation der Nutzung von KI-Tools

Über den Einsatz von Hilfsmitteln der generativen Künstlichen Intelligenz wie Chatbots, Bildgeneratoren, Programmieranwendungen, Paraphrasier- oder Übersetzungstools dokumentiere ich, dass ich Hilfsmittel der generativen Künstlichen Intelligenz verwendet habe,

- um Literatur zu finden.
- um Teile der Arbeit Korrektur zu lesen und den sprachlichen Stil zu verbessern.

Inhaltsverzeichnis

Eidesstattliche Erklärung ii

Dokumentation der Nutzung von KI-Tools iii

Kurzfassung vi

Abstract vii

1 Einleitung 1

2 Theoretische Grundlagen 2
2.1 Objekterkennung mit YOLO . . . 2
2.2 Mehrfach-Objektverfolgung mit ByteTrack . . . 2
2.3 Bildsegmentierung mit YOLO . . . 3
2.4 Zustandsautomaten . . . 3

3 Material und Versuchsgrundlage 5
3.1 Datensätze . . . 5
3.2 Hardware und Softwareumgebung . . . 5

4 Methodik und Entwicklung 7
4.1 Gesamtarchitektur und Datenfluss . . . 7
4.2 Vorverarbeitung für Infrarot-Nachtsicht . . . 8
4.3 Training und Anpassung des Objektdetektors (YOLO26n) . . . 8
4.4 Automatisierte Generierung des Segmentierungs-Datensatzes . . . 8
4.5 Bildausschnitt-Extraktion . . . 9
4.6 Training des Segmentierungsmodells (YOLO-Segmentierung) . . . 9
4.7 Objektverfolgung und Entwicklung des Zustandsmodells . . . 9

5 Design und Implementierung 11
5.1 Softwarearchitektur und Kernkomponenten . . . 11
5.1.1 Das Modul `models.py`: Die `CatFlapPipeline` . . . 11
5.1.2 Das Modul `state_machine.py`: Zeitliche Aggregation . . . 12
5.1.3 Die Orchestrierung (`processor.py`) . . . 12
5.2 Die Verbindung zur Hardware (`catflap.py`) . . . 13
5.3 Ausführungsumgebungen . . . 13
5.3.1 Die lokale Testumgebung (`main.py`) . . . 13

5.3.2 Die Mikrocontroller Testumgebung (`pi_main.py`) 14
5.3.3 Die Reale Welt (`pi_server.py`) 14

6 Tests und Analysen **15**
6.1 Modell-Performance (Training & Validation) 15
6.1.1 Leistung der Objekterkennung 15
6.1.2 Leistung der Bildsegmentierung 18
6.2 Evaluationsmethodik und Datensatz 21
6.2.1 Testdatensatz 21
6.2.2 Annotation der Referenzdaten 22
6.2.3 Definition der System-Metriken 22
6.3 Ergebnisse der Evaluation 23
6.4 Ablationsstudien 23
6.4.1 Zustandsautomat 23
6.4.2 Detektion und Segmentierung gegenüber Detektion 24
6.4.3 Graustufen gegenüber Farbbildern 25

7 Diskussion **26**
7.1 Beantwortung der zentralen Forschungsfrage 26
7.2 Bewertung des Architektur-Ansatzes 26
7.2.1 Stärken der Verarbeitungsketten-Kaskadierung 27
7.2.2 Die Rolle der asymmetrischen Randerweiterung 27
7.3 Schwächen, Limitationen und Fehlerquellen 27
7.4 Lokale Datenverarbeitung 28

8 Zusammenfassung und Ausblick **29**
8.1 Zusammenfassung der Erkenntnisse 29
8.2 Ausblick und Weiterentwicklungsmöglichkeiten 30
8.2.1 Kontinuierliches Sammeln schwieriger Negativbeispiele 30
8.2.2 Verlust der Tiefeninformationen 30
8.2.3 Hardware-Beschleunigung 30

Quellenverzeichnis **31**
Literatur 31

Kurzfassung

Freigängerkatzen bringen regelmäßig Beutetiere wie Mäuse oder Vögel in den Wohnraum. Herkömmliche Katzenklappen können dies nicht verhindern, da sie lediglich zwischen bekannten und fremden Tieren unterscheiden. Eine kamerabasierte Beuteerkennung ist schwierig, da das Beutetier im Bild klein und oft teilweise verdeckt ist und die Vorhersagen eines Modells durch Bewegungsunschärfe und wechselnde Lichtverhältnisse von Einzelbild zu Einzelbild schwanken. Entscheidungen auf Basis einzelner Bilder führen daher zu einem ständigen Wechsel zwischen Sperren und Freigeben der Klappe.

Die vorliegende Arbeit untersucht, ob eine Verarbeitungskette aus Objekterkennung, Objektverfolgung, Bildsegmentierung und zeitlicher Zustandsmodellierung eine robustere Beuteerkennung ermöglicht als die Auswertung einzelner Bilder. Ein feinabgestimmter YOLO-Objektdetektor lokalisiert die Gesichter der Katzen, ByteTrack ordnet die Erkennungen über die Zeit eindeutigen Identitäten zu, und ein YOLO-Segmentierungsmodell bewertet einen nach unten erweiterten Ausschnitt jedes Katzengesichts auf das Vorhandensein von Beute. Ein Zustandsautomat aggregiert die Konfidenzwerte jeder Katze über ein gleitendes Zeitfenster und entscheidet erst dann über die Freigabe der Klappe. Für den Nachtbetrieb arbeitet das System auf Graustufenbildern. Die Trainingsdaten des Segmentierungsmodells wurden mangels realer Aufnahmen mithilfe generativer KI erzeugt, und die Modelle wurden für den Betrieb auf einem Raspberry Pi 5 optimiert.

Die Evaluierung anhand realer Videosequenzen zeigt, dass das Gesamtsystem für alle Testvideos die korrekte Entscheidung trifft. In Ablationsstudien reduziert der Zustandsautomat die Anzahl der Zustandswechsel gegenüber der Einzelbildentscheidung um mehr als die Hälfte. Der zweistufige Ansatz übersieht deutlich weniger Katzen als ein einstufiger Detektor.

Die Ergebnisse bestätigen, dass die Kombination aus Objekterkennung, Objektverfolgung und zeitlicher Zustandsmodellierung deutlich stabilere Entscheidungen liefert als eine Einzelbildauswertung und sich auf kostengünstiger, lokaler Hardware umsetzen lässt.

Abstract

Outdoor cats regularly bring prey such as mice or birds into the home. Conventional cat flaps cannot prevent this, as they only distinguish between known and unknown animals. Camera-based prey detection is challenging because the prey is small and often partially occluded, and the predictions of a model fluctuate from frame to frame due to motion blur and changing lighting conditions. Decisions based on individual frames therefore lead to the flap constantly switching between locking and unlocking.

This thesis investigates whether a processing pipeline consisting of object detection, object tracking, image segmentation and temporal state modelling enables more robust prey detection than the evaluation of individual frames. A fine-tuned YOLO object detector localises the faces of the cats, ByteTrack assigns the detections to consistent identities over time, and a YOLO segmentation model examines a downward-extended crop of each cat's face for the presence of prey. A state machine aggregates the confidence scores of each cat over a sliding time window and only then decides whether to unlock the flap. To support operation at night, the system works on grayscale images. Due to the lack of real recordings, the training data for the segmentation model was created using generative AI, and the models were optimised for operation on a Raspberry Pi 5.

The evaluation on real video sequences shows that the overall system makes the correct decision for all test videos. In ablation studies, the state machine reduces the number of state changes by more than half compared to frame-by-frame decisions. The two-stage approach misses considerably fewer cats than a single-stage detector.

The results confirm that the combination of object detection, object tracking and temporal state modelling leads to considerably more stable decisions than frame-by-frame evaluation and can be implemented on low-cost local hardware.

Kapitel 1

Einleitung

Freigänger-Katzen bringen von Zeit zu Zeit Beutetiere, wie z.b. Mäuse oder Vögel, mit ins Haus, was für den Besitzer sehr unangenehm sein kann. Diese Beutetiere sind auf Kameraaufnahmen oft schwer erkennbar, da sie klein sind oder verdeckt werden. Wenn man versucht, dieses Problem mit reiner Objekterkennung zu lösen, erweisen sich reine Einzelbilder oft als fehleranfällig [0]. Bewegungsunschärfe oder Verdeckungen führen dazu, dass die Beute nicht mit der benötigten Genauigkeit erkannt werden kann. Eine Momentaufnahme ist hier nicht ausreichend, weshalb eine Analyse über mehrere Einzelbilder erforderlich ist.

Diese Bachelorarbeit beschäftigt sich mit der durch künstliche Intelligenz gestützter Erkennung von Beutetieren im Maul einer Katze. Das Ziel dieses Projektes ist es, eine Alternative zu den herrkömlichen Katzenklappen zu bieten und dem Katzenbesitzer das lästige fangen von Beute im Wohnraum zu ersparen.

Da die Bilderkennung unter anderem das Herzstück dieses Projektes ist, habe ich mich intensiv mit verschiedenen Algorithmen beschäftigt. Im ersten Schritt wird das Objektdetektionsmodell YOLO26 [0] eingesetzt. YOLO26 erreicht eine hohe Genauigkeit während es auch auf Endgeräten echtzeitfähig bleibt. Anschließend kommt der ByteTrack [0] Algorithmus zum Einsatz. Dieser Algorithmus verfolgt die erkannten Begrenzungsrahmen über die Zeit und sorgt dafür, dass das System die Katze auch bei kurzzeitigen Verdeckungen nicht aus den Augen verliert. ByteTrack sorgt auch dafür, dass jede Katze eindeutig ist sollten sich mehrere Katzen auf dem Videoeinzelbild befinden. Die erkannten Bereiche werden anschließend durch eine weitere künstliche Intelligenz (YOLO-Segmentierung) ausgewertet, und in zwei Kategorien eingeteilt, Katze mit und ohne Beute. Um die Zuverlässigkeit zu erhöhen und Fehlalarme zu vermeiden, trifft man nicht für jedes Videoeinzelbild eine Entscheidung sondern sammelt Daten über mehrere Videoeinzelbilder hinweg und trifft für diese Daten als gesamtes eine Entscheidung. Dafür wurde ein eigener Zustandsautomat (State Machine) eingebunden. Dieser wertet die Historie der Bilder über ein gleitendes Zeitfenster aus und trifft dann eine Entscheidung, ob die Katze beute im Mund trägt oder nicht. In dieser Arbeit wird evaluiert, inwiefern diese Kombination aus Objekterkennung, Segmentierung, Objektverfolgung und Zustandsautomaten eine bessere Lösung ist als die reine Bewertung durch Einzelbilder.

Kapitel 2

Theoretische Grundlagen

In diesem Kapitel werden die theoretischen Grundlagen erläutert, diese bilden dass Fundament dieser Arbeit. Es werden ausschließlich Technologien und Algorithmen erklärt, welche in der finalen Arbeit verwendet werden. Dazu zählen Objekterkennung, Objektverfolgung, Bildsegmentierung und Zustandsautomaten.

2.1 Objekterkennung mit YOLO

Die automatische Bildanalyse lässt sich grundsätzlich in verschiedene Problemklassen unterteilen [0]. Während die reine Bildklassifikation lediglich die Frage beantwortet, ob ein bestimmtes Objekt in einem Bild vorhanden ist, geht die Objekterkennung einen Schritt weiter [0]. Sie ermittelt nicht nur die Klasse des Objekts, sondern auch die genaue Position an der sich das Objekt befindet.

In dieser Arbeit wird YOLO [0] in der Version 26 verwendet [0]. Im Gegensatz zu zweistufigen Detektoren, welche zuerst interessante Bildregionen extrahieren und erst im zweiten Schritt klassifizieren [0], ist YOLO ein einstufiger Detektor, welcher das gesamte Bild in einem Durchlauf mithilfe eines Faltungsnetzwerk [0] analysiert [0]. Das Bild wird in Raster unterteilt, jede Rasterzelle gibt das Netzwerk mehrere Begrenzungsrahmen und die zugehörigen Klassenwahrscheinlichkeiten und Konfidenzwerte voraus [0]. Der Begrenzungsrahmen ist der Rahmen in dem sich ein Objekt befindet. Die Klassenwahrscheinlichkeit gibt an wie wahrscheinlich das erkannte Objekt zur Klasse Katze gehört. Der Konfidenzwert hingegen gibt an, wie sicher sich das Netzwerk ist, dass es sich überhaupt um ein Objekt handelt [0].

Für diese Arbeit ist YOLO unerlässlich, da ein solches System niedrige Inferrenzzeiten benötigt [0]. Das ist auf der einen Seite wichtig damit das System verlässlich bleibt und die Katze nicht zu lange vor der Klappe warten muss. Das Modell liefert für jede erkannte Katze die exakten Koordinaten dessen Gesichtes, was die Basis für die Objektverfolgung bildet.

2.2 Mehrfach-Objektverfolgung mit ByteTrack

Eine Objekterkennung wie YOLO alleine bewertet jedes Videoeinzelbild isoliert. Wenn sich eine Katze auf einem Video befindet, dann wird jedes Einzelbild des Videos separat

betrachtet. Jede erkannte Katze ist also eine neue Katze. Durch eine Objektverfolgung wird es möglich, eine Katze über ein ganzes Video eindeutig zuzuordnen. Eine Mehrfach-Objektverfolgung ist dann die Erweiterung auf mehrere Katzen pro Video [0]. Das ist zwingend erforderlich um eine zeitliche Analyse einer bestimmten Katze durchführen zu können.

In dieser Arbeit erzeugt ein Objektdetektor (in diesem Fall YOLO26) die Begrenzungsrahmen, diese enthalten die genauen Koordinaten der Objekte, in unserem Fall vom Gesicht der Katze. Im zweiten Schritt übernimmt der Tracker. Er versucht die neuen Erkennungen den bekannten zuzuordnen, sollte das nicht möglich sein wird eine neue Identität erzeugt [0]. Im Fall von ByteTrack [0] passiert dieses zuordnen durch Bewegungsvorhersagen welche der Kalman-Filter [0] trifft.

ByteTrack hebt sich von klassischen Trackern [0] ab indem er auch Erkennungen mit niedrigem Konfidenzwert verwendet um Identitätswechsel vorzubeugen, damit dies nicht zu Fehlalarmen führt, ordnet er im ersten Schritt die Begrenzungsrahmen mit hohen Konfidenzwerten zu [0]. Anschließend werden die Objekte mit den niedrigen Konfidenzwerten verglichen welche im vorigen Schritt nicht zugeordnet werden konnten. Dadurch wird die Verfolgung robuster bei Verdeckungen [0]. Daher eignet sich ByteTrack ideal für unvorhersehbare Bewegungen und Okklusionen beim Anwendungsfall einer Katzenklappe.

2.3 Bildsegmentierung mit YOLO

Die Segmentierung arbeitet wie die Objekterkennung nach dem Prinzip von Faltungsnetzwerken [0]. Moderne YOLO-Versionen, wie in unserem Fall YOLO26, bieten eben nicht nur die Objekterkennung, sondern auch die Segmentierung an [0]. Diese Netzwerke sind durch ihre Architektur für schnelle Inferenz bei hoher Genauigkeit ausgelegt [0].

Während die Objekterkennung uns zeigt wo sich eine Katze befindet, wissen wir aber noch nicht ob eine Katze Beute im Mund trägt oder nicht. Hier kommt die Bildsegmentierung zum Einsatz [0]. Dieser Ansatz hat den Vorteil dass die Segmentierung nicht mehr das ganze Bild verstehen muss, das Modell bekommt lediglich das Gesicht der Katze, dieser Ansatz benötigt einen wesentlich kleineren Datensatz, da die Segmentierung immer eine ähnliche Aufgabe mit wenig Varianz bekommt.

Die Aufgabe des Segmentierungsmodells ist somit eine binäre Entscheidung, trägt die Katze Beute im Mund oder nicht. Die Ausgabe des Modells ist ein Konfidenzwert wie wahrscheinlich die Katze Beute trägt.

2.4 Zustandsautomaten

Während der Objektdetektor und die Segmentierung Momentaufnahmen liefern, welche durchaus schwanken können, hilft uns bei der Entscheidung ein Zustandsautomat. Er aggregiert die Ergebnisse von Detektor und Segmentierung zeitlich, dadurch wird das Ergebnis nicht stark durch Ausreißer beeinflusst, und wird robuster gegen Fehler [0].

Ein endlicher Zustandsautomat ist ein Modell, welches eine endliche Menge an Zuständen beinhaltet [0]. Das Modell befindet sich zu einem Zeitpunkt immer in genau einem Zustand. Durch bestimmte Ereignisse oder erfüllen bestimmter Bedingungen,

wird der aktuelle Zustand des Automaten bestimmt, z.B. wenn das System die letzten 5 mal Katze mit Maus erkannt hat, dann wechselt der Zustandsautomat auf Zugang verweigert [0]. Der Übergang von einem Zustand löst für gewöhnlich auch Aktionen aus, in unserem Fall würden wir die Katzenklappe öffnen oder schließen [0].

Ein einfaches Beispiel für einen Zustandsautomaten ist eine Ampel, das Modell repräsentiert ob die Ampel Rot, Gelb oder Grün ist. Bedingung für das Wechseln ist ein Timer. Die Aktion wäre das Umschalten der Ampel.

In unserem Kontext dient der Zustandsautomat Ungenauigkeiten der Objekterkennung und Segmentierung zu glätten. Anstatt bei einem Ausreißer sofort den Zustand der Katzenklappe zu ändern, wird der Zustandsübergang an Bedingungen (z.b. einen Schwellenwert über ein Zeitfenster) gebunden.

Kapitel 3

Material und Versuchsgrundlage

In diesem Kapitel werden die konkreten Ressourcen und Technologien erläutert, welche für das Training der Modelle und die Evaluation der Verarbeitungskette verwendet wurden.

3.1 Datensätze

Für die Objekterkennung und Bildsegmentierung wurden unterschiedliche Datensätze benötigt, da beide ein spezifisches Problem lösen.

Datensatz für die Objekterkennung (Katzengesichter):
Für das Training der Objekterkennung mit YOLO26 wurde ein öffentlicher Kaggle-Datensatz verwendet („Cat Dataset“ von Crawford [0]). In diesem Datensatz ist bereits für jede Katze gekennzeichnet wo sich dessen Gesicht im Bild befindet, außerdem enthält er eine große Variation an Katzen in verschiedensten Lichtverhältnissen, Posen und Hintergründen. Die Daten mussten nur noch leicht umgeändert werden damit sie für das YOLO-Training geeignet waren.

Datensatz für die Segmentierung (Beutetiere):
Im Gegensatz zur Objekterkennung wo es bereits einen fertigen Datensatz gab, fand ich zu diesem Problem kaum Daten. Entweder waren Datensätze quantitativ oder qualitativ zu niederwertig. Um dieses Problem zu lösen wurden die Trainingsdaten mit hilfe von KI-generiert. Dadurch konnten gezielt verschiedene Szenarien, sprich unterschiedliche Beutetiere und Posen, erzeugt werden. Durch diese Art und Weise konnte das YOLO26 Modell mit ausreichender Genauigkeit trainiert werden.

3.2 Hardware und Softwareumgebung

Das Training der Modelle benötigt im Vergleich zur reinen Verwendung sehr ressourcenintensiv, daher habe ich das Training ausgelagert um schnell an Ergebnisse zu kommen.

Hardware-Setup:
Das Training der Modell wurde auf der Plattform Kaggle durchgeführt, diese stellt eine ausreichend starke GPU zur Verfügung welche für das Training unerlässlich war. Die späteren Tests wurden zuerst auf meiner lokalen Systemarchitektur und anschließend noch auf einem Mikrocontroller, in diesem Fall auf dem Raspberry Pi 5, getestet.

Softwareumgebung:
Die Implementierung des Systems erfolgte mit Hilfe der Programmiersprache Python [0]. Für Detektor und Segmentierungsmodell wurde das `ultralytics`-Paket verwendet. Bilderverarbeitungsoperationen wie z.b. Extrahieren von Bildausschnitten, wurden mit OpenCV [0] realisiert. Für die optimale Inferrenzgeschwindigkeit wurden die Modelle in das TensorFlow Lite [0] Format exportiert, da diese extra für ARM [0] Architekturen optimiert sind, welche von unserem Mikrocontroller verwendet wird.

Kapitel 4

Methodik und Entwicklung

In diesem Kapitel wird die konzeptionelle Entwicklung und technische Umsetzung des vorgestellten Erkennungssystems dieser Arbeit erläutert. Es wird der Weg von der Problemstellung bis hin zur Lösung beschrieben. Dabei wird genauer auf die Designentscheidungen und die Architektur der einzelnen Komponenten der Verarbeitungskette eingegangen.

4.1 Gesamtarchitektur und Datenfluss

Um zuverlässig beurteilen zu können ob eine Katze ein Beutetier im Maul trägt, reicht weder eine reine Objekterkennung noch eine reine Bildsegmentierung aus. Da wir nur einen begrenzten Datensatz an Katzen mit Beutetieren zur Verfügung haben, ist es unerlässlich diese Aufgabe auf mehrere Stufen zu teilen [**baumann2020smartcatflap**]. Daraus ergibt sich eine Verarbeitungskette aus Objekterkennung, Objektverfolgung, Bildsegmentierung und zeitlicher Zustandsmodellierung. Dabei sieht die finale Verarbeitungskette wie folgt aus:

1. Ein Einzelbild der Kamera wird aufgenommen.
2. Die YOLO Objekterkennung analysiert das Bild.
   (a) Wenn keine Katze gefunden wurde, endet die Kette hier.
   (b) Wird mindestens eine Katze gefunden, so wird jedes Katzengesicht aus dem Bild extrahiert.
3. Der ByteTrack-Algorithmus ordnet den Katzengesichtern eine Identität zu oder erstellt eine neue, sollte die Katze frisch ins Bild gekommen sein.
4. Das YOLO-System untersucht jedes Katzengesicht einzeln. Danach meldet es für jede Katze entweder ‚sauber' oder ‚Beute'.
5. Der Zustandsautomat aggregiert die Ergebnisse der Bildsegmentierung über die Zeit und trifft dann eine finale Entscheidung, in welchem Zustand (z. B. „Katze mit Beute") sich die verfolgten Objekte befindet.
   (a) Trägt eine Katze Beute im Maul so wird die Klappe geschlossen.
   (b) Sind alle Katzen sauber so wird die Klappe geöffnet.

4.2 Vorverarbeitung für Infrarot-Nachtsicht

Da Katzen ihre Beute nicht nur am Tag bei optimaler Belichtung ins Haus bringen, ist eine zentrale Anforderung an das System das es auch Nachts und bei schlechter Belichtung funktioniert. Das bringt eine weitere Schwierigkeit mit sich: Nachtsichtkameras schalten bei Dunkelheit auf Infrarot um, wodurch Graustufenbilder entstehen.

Damit das neuronale Netz die Farbinformationen nicht erlernt, werden alle Bilder vor dem Training und der Inferenz zu Graustufenbildern konvertiert. Andernfalls würde weder Objekterkennung noch Bildsegmentierung bei Nacht zuverlässig funktionieren.

4.3 Training und Anpassung des Objektdetektors (YOLO26n)

Für die Objekterkennung kommt die Archtitektur YOLO26 in der Ausführung `yolo26n` zum Einsatz. Sie ist die kleinste und bietet hohe Erkennungsgenauigkeit während es die notwendige Inferenzgeschwindigkeit für Mikrocontroller liefert.

Standardmodelle von YOLO26 sind nicht spezifisch auf die Erkennung von Katzengesichtern abgestimmt. Daher wurde eine Feinabstimmung für genau dieses Problem durchgeführt [0]. Grundlage hierfür ist der Kaggle-Datensatz (Cat Dataset von Crawford [0]), da dieser bereits Annotationen für das Gesicht enthält. Der Datensatz beinhaltet die Koordinaten für Augen, Ohren und Mund. Um diese für YOLO nutzbar zu machen, müssen sie in ein Begrenzungsrahmen-Format umgewandelt werden. Dafür wurde ein Algorithmus entwickelt, der die Breite des Rahmens durch die Ohren und die Höhe durch die Ohren und den Mund vorgibt.

Die Feinabstimmung des YOLO26n-Modells erfolgte über 100 Epochen und einer Bildauflösung von 640x640 Pixeln und der Standard YOLO-Augmentation [0]. Um eine Überanpassung des Modells zu vermeiden, wird das Training abgebrochen, sollte in den letzten zehn Epochen keine Verbesserung mehr eintreten. [0].

4.4 Automatisierte Generierung des Segmentierungs-Datensatzes

Für die Segmentierung, ob die Katze Beute im Maul trägt, gab es leider keinen brauchbaren Datensatz. Für das Training werden zwei Arten von Bildern benötigt, Katzen mit und Katzen ohne Beute. Die Katzen ohne Beute können wir wieder aus dem Kaggle-Datensatz (Cat Dataset von Crawford [0]) verwenden. Katzen mit Beute auf der anderen Seite sind extrem schwierig zu bekommen, entweder sie sind qualitativ minderwertig oder Urheberrechtlich geschützt.

Die Bilder von Katzen mit Beute, lies ich mir daher von Gemini [0] generieren. Mithilfe der offiziellen Gemini Application Programming Interface (API), entwickelte ich einen Algorithmus, der unter Berücksichtigung des Benutzungslimits automatisch Bilder von Katzen mit Beute in unterschiedlichen Posen und Entfernungen generiert.

Für die Vorbereitung des Datensatzes zur Feinabstimmung des YOLO26n Segmentierungsmodells, wird der zuvor trainierte Katzengesichts-Detektor verwendet. Er erkennt und extrahiert die Katzengesichter und schlüsselt diese dann direkt auf zwei Gruppen auf, Katzen mit und ohne Beute. Dadurch sind die Bilder fertig annotiert und benötigen keine weitere Aufmerksamkeit mehr.

Das Segmentierungsmodell hat somit immer die gleiche Eingangslage: Er bekommt stets ein Katzengesicht ohne viel Hintergrund. Das bildet die perfekte Trainingsgrundlage, da die Segmentierung in der späteren Inferenz genau die gleichen Eingangsbilder erhält.

4.5 Bildausschnitt-Extraktion

Da der Eingang des Segmentierungsmodells der Ausgang des Detektors ist, müssen beide die erforderlichen Informationen erhalten. Während der Detektor am besten funktioniert, wenn er nur auf das Katzengesicht trainiert wird, benötigt er auch die Information, was sich unter dem Maul des Katzengesichts befindet.

Daher wird der Begrenzungsrahmen des Detektors verändert bevor ihn das Segmentierungsmodell bekommt. Das Katzengesicht wird nach untenhin um die doppelte Gesichtshöhe und seitlich um jeweils 80 % der Gesichtsbreite erweitert um sicher zu gehen das die Beute im Bild ist.

Dadurch das die Eingabeschicht des Segmentierungsmodells ein 3-Kanal-Bild erwartet wird die Graustufe dreifach dupliziert (`cv2.cvtColor(gray_crop, cv2.COLOR_GRAY2BGR)`).

4.6 Training des Segmentierungsmodells (YOLO-Segmentierung)

Als Segmentierungsmodell kommt wie beim Detektor ein YOLO-Modell zum Einsatz. Dadurch das wir uns bereits in der Ultralytics Umgebung befinden und moderne YOLO-Versionen hochoptimierte Netzwerke für Segmentierungsaufgaben bereitstellen, war diese Entscheidung naheliegend.

Das Modell bekommt ein 640x640 Pixel großes Bild und liefert einen Konfidenzwert, der angibt mit welcher Wahrscheinlichkeit sich Beute auf dem Bild befindet. Dadurch dass der Datensatz des Modells relativ klein ist, wird hier stark auf Datenvervielfältigung gesetzt (zufällige Rotation des Bildes, horizontale Spiegelung). Durch diese Augmentationsschritte wird das Modell robuster gegen unbekannte Posen der Katzen.

4.7 Objektverfolgung und Entwicklung des Zustandsmodells

Wenn sich eine Katze der Klappe nähert, ändert sich die Qualität der Bilder durch Bewegungsunschärfe oder ungünstige Winkel stark. Das hat zur Folge, dass die Ausgaben des Segmentierungsmodells schwanken. Wenn z.b. die Beute kurzzeitig verdeckt ist, würde das zur Folge haben das die Katze ins Haus darf.

Damit dieses Problem eingedämpft wird, wurde die Objekterkennung um den ByteTrack-Algorithmus erweitert. Dadurch bekommt eine Katze beim ersten auftreten im Bild eine eindeutige Objekt-ID, diese ID bleibt in den darauffolgenden Bildern für diese Katze gleich. Basierend auf der Eindeutigkeit jeder Katze konnte nun ein Zustandsautomat entwickelt werden. Diese Klasse `StateMachine` verwaltet für jede erkannte Katze (`track_id`) ein `TrackState`-Objekt. Das Modell kennt drei Zustände:

- `NO_CAT`: Es befindet sich kein relevantes Objekt im Sichtfeld.
- `CAT_NO_PREY`: Eine Katze wurde erkannt, trägt jedoch keine Beute.

- `CAT_WITH_PREY`: Eine Katze wurde erkannt und trägt mit hinreichender Sicherheit eine Beute.

Damit nun die Systementscheidung nicht durch das aktuellste Einzelbild getroffen wird, speichert das `TrackState`-Objekt eine Historie der Konfidenzwerte des Segmentierungsmodells der letzten 15 Bilder. Der aktuelle Zustand wird bei jedem Bild neu berechnet, dabei wird einfach der gleitende Durchschnitt der Historie berechnet. Wird der Schwellenwert von 0,8 überschritten, so wird die Katze als `CAT_WITH_PREY` markiert. Fällt der Wert darunter wird sie als `CAT_NO_PREY` markiert.

Der Schwellenwert von 0,8 wurde bewusst so hoch gewählt, um zu vermeiden, dass die Katze zu Unrecht draußen bleiben muss. Dieser Wert ist aber nicht in Stein gemeißelt. Wenn es für den Endanwender wichtiger wäre, Beute im Haus um jeden Preis zu vermeiden würde man den Wert niedriger ansetzen, beispielsweise bei 0,3.

Um Speicherlecks in der Zustandsmaschine zu verhindern, werden Katzen die das Bildfeld verlassen haben oder die länger als 30 Bilder nicht mehr gefunden werden (`max_missed_frames`) aus dem Speicher entfernt.

Kapitel 5

Design und Implementierung

Nachdem sich das vorherige Kapitel mit der Methodik beschäftigt hat, geht dieses Kapitel auf die konkrete Realisierung des Systems ein. Dabei wird erläutert, wie die Architektur aufgebaut ist und wie die einzelnen Softwarekomponenten ineinandergreifen. Darüber hinaus wird auf die verwendeten Datenstrukturen eingegangen.

5.1 Softwarearchitektur und Kernkomponenten

Die Implementierung erfolgte in der Programmiersprache Python und hält sich streng an die Regeln der objektorientierten Programmierung, damit das System wart- und erweiterbar bleibt. Dabei wurde das Kernsystem in drei wesentliche Skripte unterteilt: Erkennungsmodelle (`models.py`), Zustandsautomat (`state_machine.py`) und die Orchestrierung (`processor.py`).

5.1.1 Das Modul `models.py`: Die `CatFlapPipeline`

Die Klasse `CatFlapPipeline` verwaltet Detektor und Segmentierungsmodell und vereinfacht deren Verwendung. Beim Initialisieren des Objekts werden beide Modelle geladen. Dabei werden die Pfade (absolut oder relativ) zu den Modellen ergänzt.

Die Methode `run_detector` konvertiert das Bild in Graustufen. Anschließend wird die Objekterkennung aufgerufen, durch den Parameter `persist=True` wird der in Ultralytics integrierte ByteTrack-Tracker aktiviert, welcher uns die Begrenzungsrahmen samt Objekt-IDs liefert.

Die Methode `run_prey_detector` schneidet das Bild wie in Kapitel 4.5 beschrieben zusammen. Im folgenden Codeausschnitt ist zu sehen, wie dieses Zuschneiden implementiert ist und welche Auswirkungen es auf das Katzengesicht hat. Durch das Erweitern nach unten stellen wir sicher, dass das Modell genügend Informationen über mögliche Beute im Maul der Katze erhält.

Listing 5.1: Asymmetrische Randerweiterung für den Bildausschnitt

```
1 x1, y1, x2, y2 = map(int, box)
2 # Add padding (heavy on bottom for dangling prey)
3 face_w = x2 - x1
4 face_h = y2 - y1
5 pad_w = int(face_w * 0.8)
```

```
pad_top = 0
pad_bottom = int(face_h * 2.0)
x1 -= pad_w
y1 -= pad_top
x2 += pad_w
y2 += pad_bottom
```

Anschließend wird der Bildausschnitt des Katzengesichtes vom Segmentierungsmodell verarbeitet, dieser liefert uns dann den Konfidenzwert für die Klasse `CAT_WITH_PREY`.

5.1.2 Das Modul `state_machine.py`: Zeitliche Aggregation

Der Zustandsautomat ist durch die Klasse `StateMachine` abgebildet. Der Status einer einzelnen Katze wird in der Klasse `TrackState` gespeichert, diese speichert die Historie der Konfidenzwerte in einer `collections.deque` mit einer fixen Maximallänge (`history_length = 15`). Durch diese Datenstruktur wird sichergestellt, dass beim Erreichen der Kapazitätsgrenze der älteste Wert aus der Warteschlange entfernt wird („First-In-First-Out“). Dabei bleibt das manuelle Speicher-Management komplett erspart.

Der folgende Codeausschnitt zeigt die Kernlogik der `update`-Methode. Diese berechnet bei jedem neuen Bild den gleitenden Durchschnitt (`numpy.mean`) und schaltet anschließend den Zustand entsprechend um.

Listing 5.2: Zustandsermittlung mittels gleitendem Durchschnitt

```
def update(self, prey_confidence, frame_idx):
    self.confidence_history.append(prey_confidence)
    self.last_seen = frame_idx

    # Calculate moving average
    avg_confidence = np.mean(self.confidence_history)

    # Determine new state based on aggregated confidence
    if avg_confidence >= self.threshold:
        self.current_state = State.CAT_WITH_PREY
    else:
        self.current_state = State.CAT_NO_PREY
```

Die Klasse `StateMachine` verwaltet die verschiedenen `TrackState`-Objekte in einem Dictionary (`track_id` → `TrackState`). Die Methode `cleanup_stale_tracks` iteriert regelmäßig über dieses Dictionary und entfernt die Objekte, die seit mehr als 30 Einzelbildern nicht mehr vom Tracker zugeordnet wurden konnten, um Speicherlecks zu vermeiden.

Sobald eine oder mehrere Katzen mit Beute im Maul erkannt werden, wirft das `StateMachine`-Objekt ein Ereignis und meldet, dass der Eintritt von Katzen nicht erlaubt ist. Andernfalls, d. h. wenn alle Katzen sauber sind, meldet das Ereignis, dass der Eintritt erlaubt ist.

5.1.3 Die Orchestrierung (`processor.py`)

Im Skript `processor.py` geschieht die tatsächliche Verarbeitung der Einzelbilder. Die Klasse `CatFlapProcessor` verwaltet ein `StateMachine` Objekt, zählt wie viele Bilder

pro Sekunde verarbeitet werden können und bietet die Methode `process_frame` an. Diese bildet das Herzstück der Klasse, sie bekommt ein Videoeinzelbild als Parameter übergeben, danach wird über alle erkannten Katzen iteriert, dabei wird für jedes Objekt die Segmentierung durchgeführt. Im letzten Schritt wird dann der Zustandsautomat aktualisiert und die Visualisierungen (Begrenzungsrahmen und Zustand) werden auf das Bild eingezeichnet.

Wie schwierig es ist, qualitativ hochwertige Daten von Katzen mit Beute zu sammeln, wurde im Kapitel 3 bereits erwähnt. Um das Segmentierungsmodell gezielt zu verbessern, wurde ein Mechanismus implementiert, der Bilder speichert, bei denen das Modell unsicher war (Konfidenzwert zwischen 0,3 und 0,7). Dadurch können die Szenarien, die dem Modell am schwersten fallen, gezielt verbessert werden. Dieses Feature wird mit dem Kommandozeilenparameter `-save_uncertain` aktiviert. Um Speicher zu schonen, wird nach Auslösung des Mechanismus die Speicherung derselben `track_id` für 30 Einzelbilder gesperrt.

5.2 Die Verbindung zur Hardware (`catflap.py`)

Die im Skript `catflap.py` enthaltene Klasse `Catflap` stellt dabei die Hardwareumgebung zur Verfügung. Durch Polymorphismus können verschiedene Arten erzeugt werden. Dabei liefert die Methode `capture_continuous` das nächste Bild der Kamera. Außerdem bietet die Klasse Methoden zum schließen `lock_in` / `lock_out` und öffnen `unlock_in` / `unlock_out` der Klappe.

- `lock_in`: Katze darf Haus nicht betreten.
- `lock_out`: Katze darf Haus nicht verlassen.
- `unlock_in` und `unlock_out`: Heben die jeweilige Sperre auf.

Die Ableitung `SimulationCatflap` wird zur Simulation verwendet, die Videoeinzelbilder liefert hier ein Video, sperren und entsperren der Klappe wird durch eine Ausgabe auf die Konsole simuliert. Für den realen Betrieb gibt es die Klassen `ServoCatflap`, die mithilfe eines Servo-Motors sperrt, und `SureCatflap`, die als Anbindung zu einer vorhandenen SureFlap-Katzenklappe [0] verwendet werden kann.

5.3 Ausführungsumgebungen

Das System kann je nach Einsatzzweck in verschiedenen Umgebungen ausgeführt werden, die jeweils eigene Skripte besitzen.

5.3.1 Die lokale Testumgebung (`main.py`)

Das Skript `main.py` wurde entwickelt um neu trainierte Modelle schnell auf der lokalen Umgebung (z.b. Windows Laptop) validieren zu können. Dabei liest das Skript eine Videodatei ein. Danach durchläuft jedes Videoeinzelbild die `process_frame` Methode der Klasse `CatFlapProcessor`. Anschließend wird die Visualisierung auf das Video eingezeichnet und abgespielt.

5.3.2 Die Mikrocontroller Testumgebung (`pi_main.py`)

Um das Zusammenspiel aller Komponenten sinnvoll auf dem Endgerät testen zu können wurde das `pi_main.py` Skript entwickeltentwickelt. Dadurch ist es möglich, jeden Teil – also Detektor, Segmentierungsmodell, Servo und Kamera – isoliert zu testen. Außerdem ist dieser teil wichtig zum zu testen wie viele Bilder pro Sekunde der Raspberry Pi 5 verarbeiten kann. Ansonsten funktioniert das Skript gleich wie `main.py`.

5.3.3 Die Reale Welt (`pi_server.py`)

Für den realen Betrieb reichen die vorher erwähnten Skripte nicht aus. Damit man während des Betriebes die Klappe manuell sperren / entsperren und die aktuellen Kamera-Aufnahmen einsehen kann, werden im Skript `pi_main.py` mithilfe des `flask`-Paketes diese Funktionen als HTTP Endpunkte zur Verfügung gestellt.

Auch hier wird ein `CatFlapProcessor`-Objekt erzeugt und verwaltet, dieses bekommt kontinuierlich die Einzelaufnahmen der Kamera. Sollte sich der Zustand des darin enthaltenen `TrackState`-Objekts ändern, wird, wie in Kapitel 5.1.2 beschrieben, ein Ereignis geworfen. Sind alle Katzen sauber, wird die Klappe geöffnet, ansonsten wird sie gesperrt. Wurde die Katzenklappe entsperrt, so startet ein Zeitmesser, der die Klappe nach 30 Sekunden wieder schließt.

Kapitel 6

Tests und Analysen

In diesem Kapitel wird die Leistungsfähigkeit des gesamten Systems evaluiert. Die Beurteilung erfolgt anhand von KI-generierten Videodaten, die dem System während der Trainingsphase nicht präsentiert wurden. Im Fokus steht dabei die Frage, ob die Verbesserungen durch zeitliche Aggregation und die zweistufige Verarbeitungskette zuverlässiger sind als die alleinige Verwendung der Objekterkennung und Segmentierung. Dabei ist es interessant zu sehen, in welchen Szenarien diese Zusätze das System deutlich beeinflussen.

6.1 Modell-Performance (Training & Validation)

Da die YOLO-Modelle einen wesentlichen Teil der Zuverlässigkeit des Systems beeinflussen, werfen wir zuerst einen Blick auf deren isolierte Leistung.

In diesem Unterkapitel wird vor allem auf die Trainingsmetriken eingegangen. Besonders Interessant sind dabei die Verlustfunktionen, welche angeben wie viele Fehler auf den jeweiligen Datensatz (Training oder Validierung) passiert sind. Die Informationen der Wahrheitsmatrizzen geben dabei Einsicht wie viele Objekte erkannt, fälschlicherweise erkannt oder vom Modell erfunden wurden.

6.1.1 Leistung der Objekterkennung

Die Trainingsmetriken (Abbildung 6.1) für den Katzengesicht-Detektor, zeigen einen kontinuierlich sinkenden Trainingsfehler (*engl. Loss*) und eine hohe Genauigkeit (*engl. Precision*) in der Validierung im Verlauf von 100 Epochen.

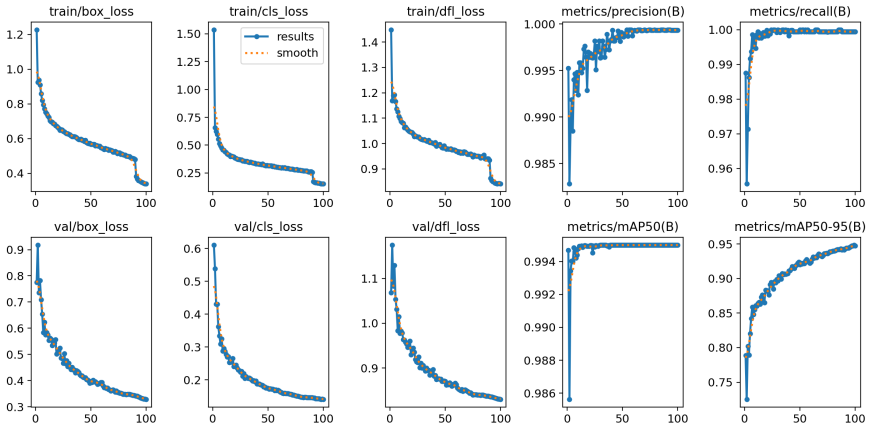


Abbildung 6.1: Trainingsverlauf und Metriken des YOLO-Objektdetektors.

Die Wahrheitsmatrix (*engl. Confusion Matrix*) der Objekterkennung (Abbildung 6.2) verdeutlicht die Präzision bei der Lokalisierung. Sie zeigt, dass bei der Validierung 1.676 Katzengesichter korrekt erkannt wurden. Das Modell hat jedoch zwei Katzengesichter erkannt, obwohl dort keines war, und eines hat es komplett übersehen. Die Matrix belegt also dass der Detektor nahezu fehlerfrei auf den Validierungsdaten funktionierte.

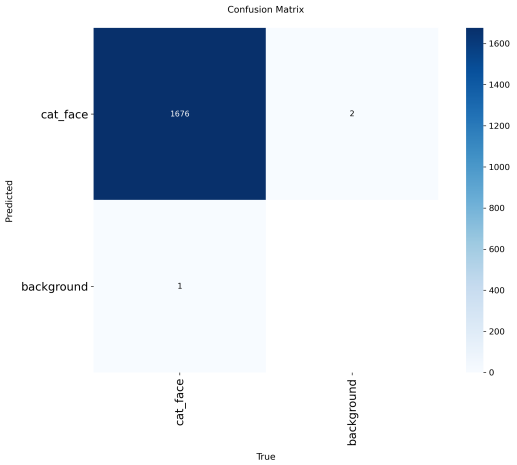


Abbildung 6.2: Wahrheitsmatrix (Detektor).

Eine wichtige Metrik, welche auch auf den Validierungsdaten beruht, ist die Präzision-Trefferquote-Kurve (*engl. Precision-Recall-Curve*), sie gibt das Verhältnis zwischen richtig erkannten Objekten und Fehlalarmen an wie sicher das Modell ist [0]. In diesem Fall zeigt der Verlauf, dass das Modell extrem sicher ist und nicht raten muss, wenn es ein Katzengesicht sieht.

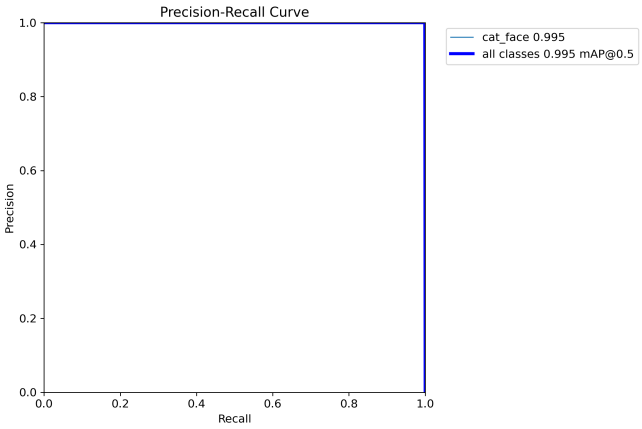


Abbildung 6.3: Präzision-Trefferquote-Kurve (*engl. Precision-Recall-Curve*).

Die F1-Maß-Kurve (Abbildung 6.4) zeigt wie sich die Leistung des Modells bei Änderung des Konfidenzschwellenwerts [0] verändert. Dadurch können wir den optimalen Schwellenwert für den realen Betrieb des Modells ermitteln. Da die Kurve bis $\approx 0,85$ flach bleibt, d.h. hier wird keine Katze übersehen, fällt sie $>$ 0,85 stark ab. Das zeigt, dass ab diesem Schwellenwert auch reale Katzen leicht übersehen werden. In der Praxis zeigt sich, dass bei einem Konfidenzschwellwert von $\approx 0,75$ ein gutes Verhältnis zwischen „Katze wird nicht erkannt“ und „es wird eine Katze erkannt, wo sich keine befindet“ entsteht.

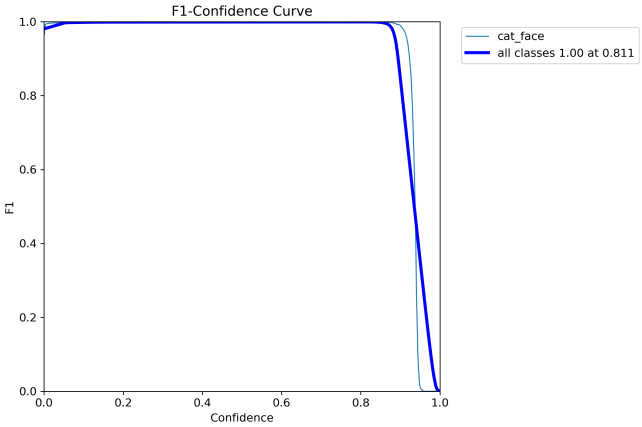


Abbildung 6.4: F1-Maß-Kurve.

Die Abbildung 6.5 liefert einen Eindruck auf die Erkennungsleistung des Modells, wenn man es auf die Validierungsdaten ansetzt.

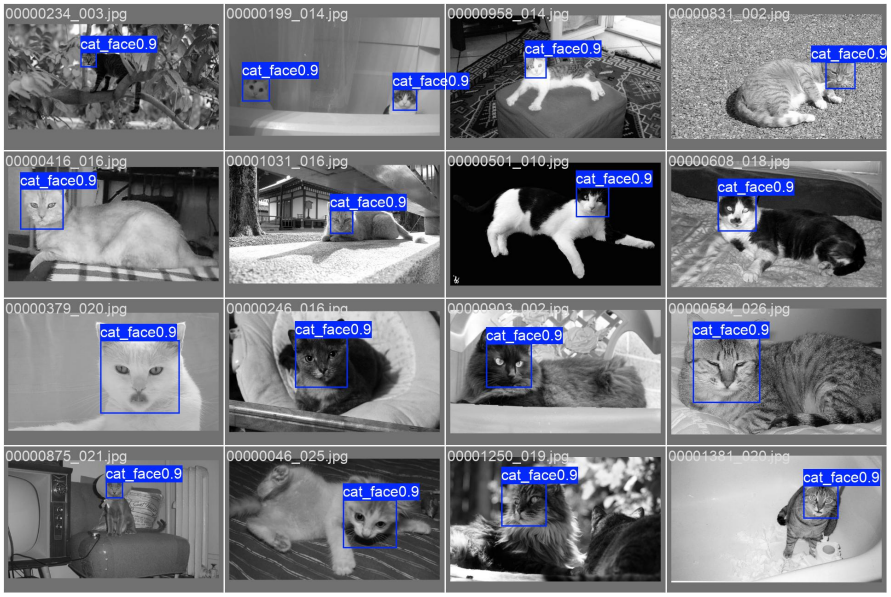


Abbildung 6.5: Vorhersagen des Detektors auf dem Validierungsdatensatz.

6.1.2 Leistung der Bildsegmentierung

Die folgenden Trainingsmetriken des Beute-Segmentierungsmodells zeigen, dass das Modell seine Aufgabe erfolgreich gelernt hat. Beim betrachten der Verlustfunktionen wird ersichtlich, dass das Modell über die Epochen weniger Fehler sowohl bei Training als auch Validierung macht, daraus können wir schließen, dass das Segmentierungsmodell lernt ohne sich dabei zu viel auf den Trainingsdatensatz anpasst.

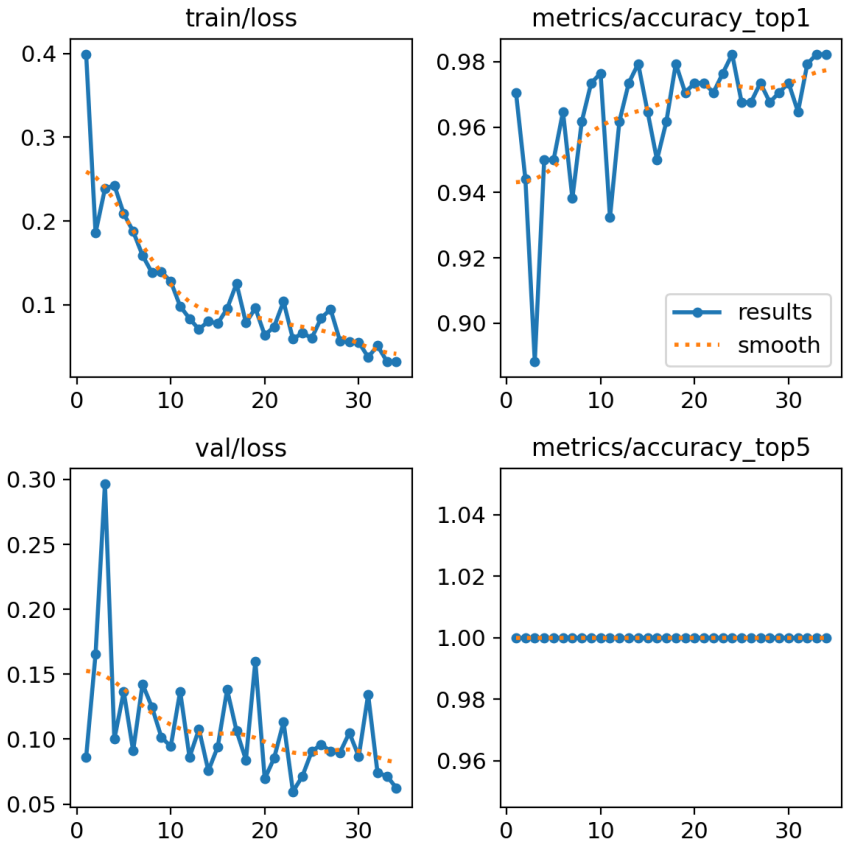


Abbildung 6.6: Trainingsverlauf und Metriken des YOLO-Segmentierungsmodells.

Die Präzision-Trefferquote-Kurve der Segmentierungsmasken (Abbildung 6.7) zeigt, dass das Modell bis zu einer Trefferquote von etwa 94 % keine einzige Fehlerkennung liefert. Auf dem Validierungsdatensatz erreicht es eine mAP50 von 0,945.

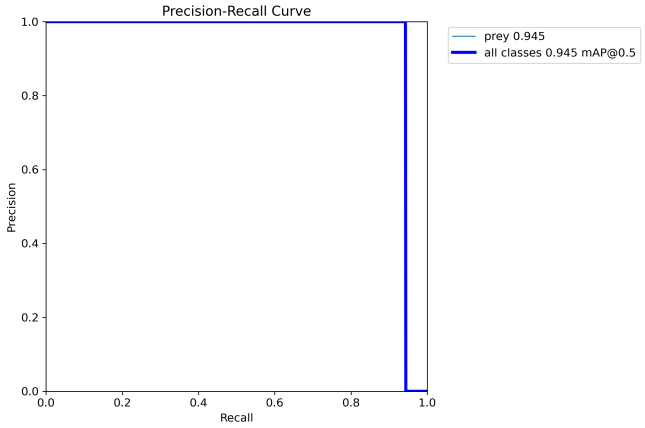


Abbildung 6.7: Präzision-Trefferquote-Kurve der Segmentierungsmasken.

Die F1-Maß-Kurve (Abbildung 6.8) erreicht ihr Maximum von 0,97 bei einem Konfidenzschwellenwert von 0,728. Zwischen etwa 0,1 und 0,85 bleibt sie nahezu konstant, das Modell reagiert in diesem Bereich also kaum empfindlich auf die Wahl des Schwellenwerts.

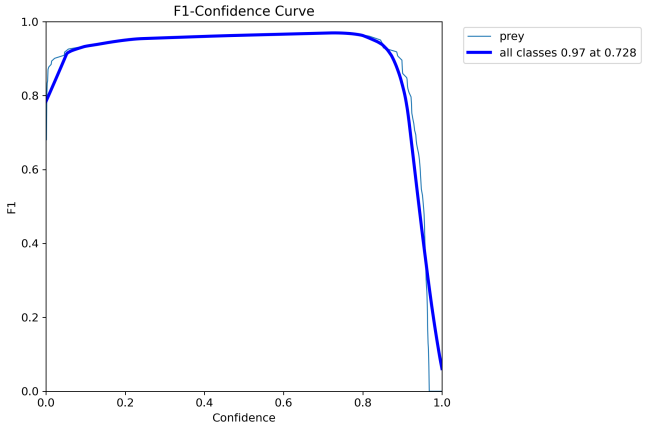


Abbildung 6.8: F1-Maß-Kurve des Segmentierungsmodells.

Mithilfe der Abbildung 6.9 lässt sich gut erkennen, wie gut das Modell mit unbekannten Daten umgeht.



Abbildung 6.9: Beispielhafte Vorhersagen des Segmentierungsmodells auf dem Validierungsdatensatz.

Rein auf dem Papier würden diese Modelle fast fehlerfreie Aussagen treffen. Die entscheidende Frage ist jedoch, wie verhält sich das System im Praxiseinsatz und welche Vorteile bietet die Verarbeitungskette gegenüber der reinen Detektion und Segmentierung.

6.2 Evaluationsmethodik und Datensatz

Für die Evaluation des Gesamtsystems wird ein eigener Testdatensatz aus KI-generierten Videosequenzen erstellt. Dieses Kapitel beschreibt den Aufbau des Datensatzes, die Annotation der Referenzdaten und die verwendeten Metriken.

6.2.1 Testdatensatz

Die Testbasis bilden zehn Videopaare, die mit dem Videomodell Veo über Gemini [0] erzeugt werden. Jedes Video zeigt die Perspektive einer fest montierten Kamera unmittelbar über der Katzenklappe, auf die sich eine oder zwei Katzen zubewegen. Ein Videopaar besteht aus einer Sequenz, in der eine Katze ein Beutetier im Maul trägt, und

einem Gegenstück mit identischer Szene und möglichst identischem Bewegungsablauf, in dem keine Katze Beute trägt. Dadurch unterscheiden sich die beiden Videos eines Paares ausschließlich im Vorhandensein der Beute.

Die Videopaare decken unterschiedliche Fellfarben, Beutetiere (Maus, Vogel, Ratte und Wühlmaus), Lichtverhältnisse (Tag, Dämmerung und Infrarot-Nachtsicht) sowie Szenen mit zwei Katzen ab. Die Videos besitzen eine Länge von zehn Sekunden bei 24 Bildern pro Sekunde. Da das System auf dem Raspberry Pi 5 rund 8 Bilder pro Sekunde verarbeitet, werden die Videos für die Evaluation auf 8 Bilder pro Sekunde ausgedünnt. Dadurch entsprechen die gemessenen Entscheidungszeiten dem realen Betrieb.

6.2.2 Annotation der Referenzdaten

Für jedes Video wird eine Referenzannotation erstellt. Sie legt für jede Katze die Zeitintervalle fest, in denen die Katze im Bild sichtbar ist, in denen ihr Gesicht erkennbar ist und in denen sie Beute trägt. Aus diesen Intervallen ergibt sich für jedes Einzelbild der korrekte Zustand der Katzenklappe: Befindet sich mindestens eine Katze mit Beute im Bild, muss die Klappe gesperrt sein, andernfalls darf sie geöffnet werden.

6.2.3 Definition der System-Metriken

Nachdem alle Videosequenzen die Verarbeitungskette durchlaufen sind, kann man die Metriken für das gesamte System erstellen. Diese leiten sich aus Wahrheitsmatrix ab:

- **Richtig-Positiv-Ergebnis (*engl. True Positive*) (TP):** Katze mit Beute darf nicht ins Haus (korrekte Sperrung).
- **Falsch-Negativ-Ergebnis (*engl. False Negative*) (FN):** Katze mit Beute darf ins Haus (Beute gelangt fälschlicherweise hinein).
- **Richtig-Negativ-Ergebnis (*engl. True Negative*) (TN):** Katze ohne Beute darf ins Haus (korrekter Durchlass).
- **Falsch-Positiv-Ergebnis (*engl. False Positive*) (FP):** Katze ohne Beute darf nicht ins Haus (Fehlalarm).

Im Kontext einer Katzenklappe besitzt die Katze aus Nutzersicht die höchste Priorität. Ein Falsch-Positiv-Ergebnis (Fehlalarm) bedeutet, dass das Haustier grundlos vor einer verschlossenen Tür steht. Um das Vertrauen des Tieres in die Klappe nicht zu zerstören, liegt der Fokus der Evaluierung primär auf der Vermeidung solcher Fehlalarme (hohe Präzision). Ein seltenes Falsch-Negativ-Ergebnis (*engl. False Negative*) (eine Maus gelangt ins Haus) wird im Tausch für eine stressfreie Nutzung der Klappe durch die Katze bewusst in Kauf genommen.

Anders als eine Bewertung auf Videoebene erfasst die bildweise Auswertung auch kurzzeitige Fehlentscheidungen. Für jedes Einzelbild mit sichtbarer Katze wird der Zustand der Klappe mit der Referenzannotation verglichen. Daraus ergeben sich folgende Kennzahlen:

- *Sicherheitsfehler*: Anteil der Einzelbilder, in denen die Klappe geöffnet ist, obwohl eine Katze mit Beute sichtbar ist.
- *Komfortfehler*: Anteil der Einzelbilder, in denen die Klappe gesperrt ist, obwohl nur saubere Katzen sichtbar sind. Gezählt wird erst nach einer Entscheidungszeit

von drei Sekunden ab dem Erkennen des Katzengesichts, da der Zustandsautomat zu Beginn bewusst sperrt.

- *Präzision*, *Trefferquote* und *F1-Maß* der Entscheidung „Klappe sperren“ über alle Einzelbilder mit sichtbarer Katze. Einzelbilder mit sauberen Katzen innerhalb der Entscheidungszeit werden dabei nicht berücksichtigt.
- *Durchgelassene Beute*: Anzahl der Videos mit Beute, in denen die Klappe mindestens einmal geöffnet wurde.
- *Entscheidungszeit*: Zeit vom Erkennen des Katzengesichts bis zur ersten korrekten Entscheidung, also bis zur Sperrung bei einer Katze mit Beute beziehungsweise bis zur Freigabe bei einer sauberen Katze.
- *Übersehene Katzen*: Anteil der Einzelbilder mit erkennbarem Katzengesicht, in denen das System keine Katze detektiert.

Die Auswertung erfolgt in zwei Schritten. Zunächst werden Detektor und Segmentierungsmodell einmal pro Video und Konfiguration ausgeführt und die Ergebnisse jedes Einzelbildes protokolliert. Anschließend wird der Zustandsautomat auf diesen Protokollen nachgebildet. Dadurch lassen sich unterschiedliche Parameter des Zustandsautomaten ohne erneute Inferenz vergleichen.

6.3 Ergebnisse der Evaluation

Metrik	Videos mit Beute	Videos ohne Beute
Sicherheitsfehler	–	–
Komfortfehler	–	–
Durchgelassene Beute	–	–
Entscheidungszeit	–	–

Tabelle 6.1: Ergebnisse des Gesamtsystems auf dem Testdatensatz.

6.4 Ablationsstudien

In den folgenden Ablationsstudien wird jeweils eine Komponente des Systems entfernt oder ersetzt, während alle übrigen Komponenten unverändert bleiben. Alle Studien verwenden den Testdatensatz und die Metriken aus Abschnitt 6.2.

6.4.1 Zustandsautomat

Um die Wirksamkeit und absolute Notwendigkeit der zeitlichen Aggregation (Zustandsautomat) zu validieren, wurde eine Ablationsstudie durchgeführt. Achtung! Einige Zustandswechsel sind gewollt: Auch wenn die Katze keine Beute trägt, wird ihr der Zugang so lange verweigert, bis sie beweist, dass sie sauber ist. Bei dieser Studie wurde das exakt selbe System zweimal auf eine Auswahl von Testvideos angewendet:

1. **Volles System:** Mit aktivem Zustandsautomaten und einem „gleitenden Durchschnitt“ (*engl. moving average*) über 15 Einzelbilder (`history_length=15`).

2. **Ablation:** Da der Zustandsautomat deaktiviert wurde, basierte die Entscheidung über die Klappensperre ausschließlich auf dem Ergebnis des aktuellen Einzelbildes. (`history_length=1`).

Zusätzlich wird eine Parameterreihe über die Länge des Zeitfensters (1, 3, 5, 8, 10, 15 und 20 Einzelbilder) und den Schwellenwert (0,3 bis 0,8) ausgewertet. Sie zeigt den Zielkonflikt zwischen Sicherheitsfehlern, Komfortfehlern und Entscheidungszeit und dient der Festlegung der finalen Parameter.

6.4.2 Detektion und Segmentierung gegenüber Detektion

Zur Validierung der Architektur unserer zweistufigen Kette (Detektor + Segmentierungsmodell) wurde für diese Studie ein einstufiger Objektdetektor (YOLO26) trainiert. Dieser besitzt zwei Klassen: „Katze mit Beute" und „Katze ohne Beute". Beide Klassen besitzen gleich viele Daten.

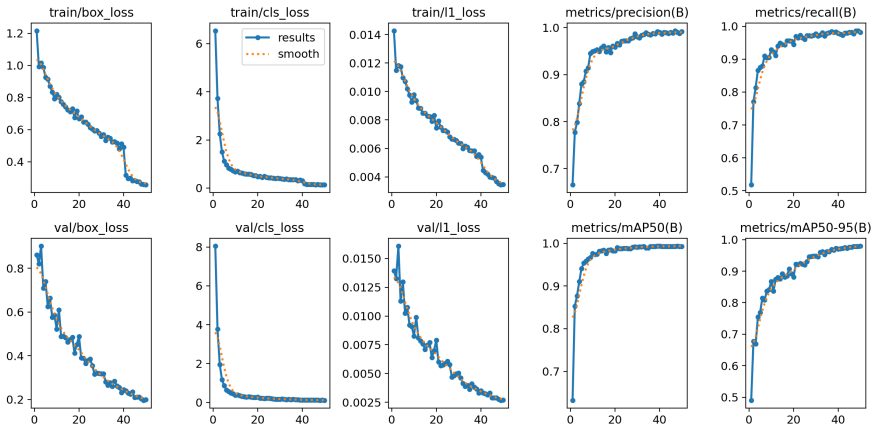


Abbildung 6.10: Trainingsmetriken des einstufigen Objektdetektors.

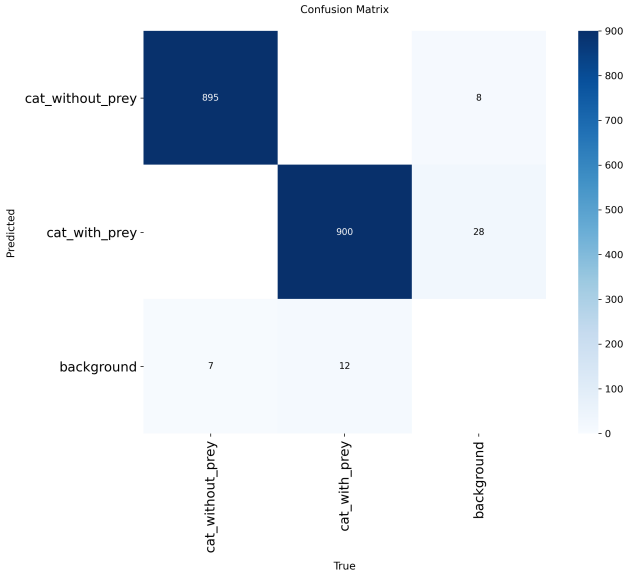


Abbildung 6.11: Konfusionsmatrix des einstufigen Objektdetektors auf den Validierungsdaten.

Um die Modelle isoliert zu vergleichen, wurde der Zustandsautomat wieder deaktiviert (`history_length=1`). Dabei wurde evaluiert wie viele Fehlklassifikationen beide Systeme trafen, sowie wie viele Katzen insgesamt im Video übersehen wurden.

6.4.3 Graustufen gegenüber Farbbildern

Da die Kamera bei Dunkelheit auf Infrarot-Nachtsicht umschaltet, arbeitet das System durchgehend auf Graustufenbildern. In dieser Studie wird ein auf Graustufenbildern trainiertes System mit einem auf Farbbildern trainierten System verglichen, getrennt nach Tag- und Nachtaufnahmen.

Kapitel 7

Diskussion

Die in Kapitel 6 präsentierten Testergebnisse liefern ein klares Bild der Systemleistung. In diesem Kapitel werden diese Ergebnisse interpretiert und kritisch bewertet. Dabei wird untersucht, inwiefern die Forschungsziele erreicht wurde, wo die grenzen des Systems liegen und welche Funktionalitäten für die Qualität ausschlaggebend waren.

7.1 Beantwortung der zentralen Forschungsfrage

Die primäre Forschungsfrage dieser Arbeit war, ob eine Verarbeitungskette aus Objekterkennung, Bildsegmentierung, Objektverfolgung und zeitlicher Zustandsmodellierung, eine robustere Beute-Erkennung ermöglicht gegenüber Einzelbildmethoden.

Die Evaluierung liefert hierauf eine eindeutig Antwort. Ein einzelbildbasierter Ansatz ist bei der Analyse von Videostreams stark anfällig für Fehler. Bewegungsunschärfe (Motion Blur), wenn die Katze ins Bild läuft, partielle Verdeckungen (Okklusionen) der Beute durch die Pfoten oder rasche Änderungen der Beleuchtung führen zu starken Ungenauigkeiten der Segmentierung. Ohne die zeitliche Aggregation im Zustandsautomaten würde die Zugangskontrolle ständig zwischen den Zuständen „verriegelt“ und „entriegelt“ wechseln (Flackern).

Durch die hilfe der `StateMachine` in Kombination mit ByteTrack wird diese Problem elegant gelöst. Die zeitliche Aggregation über eine Historie von Einzelbildern erzeugt ein robustes Signal, das Ausreißer effektiv glättet. Die Ablationsstudie „Relevanz des Zustandsautomaten“ zeigt, dass das System selbst dann richtig entscheidet, wenn einzelne Bilder falsch klassifiziert werden. Somit ist der Beweis erbracht, dass der Zustandsautomat in diesem Anwendungsfall ein wesentlicher Teil ist und zwingend für die Zuverlässigkeit im realen Betrieb ist.

7.2 Bewertung des Architektur-Ansatzes

Die Entscheidung, das Problem in zwei separate neuronale Netze (YOLO-Detektor für das Gesicht und YOLO-Segmentierungsmodell für den Zustand) zu unterteilen, hat sich als äußerst vorteilhaft erwiesen.

7.2.1 Stärken der Verarbeitungsketten-Kaskadierung

Der übliche Ansatz im Bereich maschinelles Sehen bestünde darin, ein einziges Objekterkennungsmodell darauf zu trainieren, direkt die Klassen „Katze“ und „Beutetier“ im Gesamtbild zu finden. Bei einer Katzenklappe scheitert dieser Ansatz jedoch häufig. Eine Maus im Maul einer Katze nimmt im Gesamtbild der Kamera nur einen verschwindend geringen Bruchteil der Pixel ein und ist durch das Maul und die Schnurrhaare stark verdeckt. Standard-Objektdetektoren haben immense Schwierigkeiten, derart kleine, okkludierte Objekte zuverlässig mit einem Begrenzungsrahmen zu versehen [**tong2020small**].

Der in dieser Arbeit gewählte Kaskadierungs-Ansatz umgeht dieses Problem elegant [**baumann2020smartcatflap**, **zhang2014partbased**]. Der erste Detektor muss lediglich das vergleichsweise große und markante Katzengesicht finden – eine triviale Aufgabe für moderne Architekturen wie YOLO26, was sich in der hohen Erkennungsstabilität niederschlug. Durch das anschließende Herausschneiden (Zuschneiden) dieses Bereichs wird die Aufmerksamkeit des nachgelagerten Segmentierungsmodells exklusiv auf die relevante Region fokussiert (Aufmerksamkeitsmechanismus *by design*). Das Segmentierungsmodell muss keine kleine Maus im Hintergrund des Gartens suchen, sondern analysiert lediglich die Pixelstruktur direkt am Maul der Katze.

7.2.2 Die Rolle der asymmetrischen Randerweiterung

Dadurch das die Annotationen des Datensatzes im Detektor („Cat Dataset“ von Crawford [0]) den unteren Teil des Mauls komplett verdecken, ist es unerlässlich für Training und Inferrenz den Begrenzungsrahmen nach untenhin Proportional zu erweitern.

7.3 Schwächen, Limitationen und Fehlerquellen

Aus der Analyse werden die Limitationen des aktuellen Systems genau aufgezeigt. Dadurch entsteht das Potenzial für zukünftige Verbesserungen:

1. **Verlust von Tiefeninformationen:** Dadurch das die Kamera ein 2D Bild aufnimmt kann nicht unterschieden werden, wie weit weg sich ein Objekt unterhalb des Katzenmauls befindet. Daraus folgt, dass das Modell den Unterschied zwischen Hintergrund und Beute nicht zuverlässig trennen kann.
2. **Extreme Bewegungsgeschwindigkeit:** Wenn eine Katze fluchtartig durch die Klappe rennt, bleibt sehr wenig Zeit eine klare Aussage zu treffen. Bei einer Laufzeit von ca. 8 FPS verbleiben dem System nur eine Handvoll Einzelbilder zur Analyse. In diesem Szenario kann die `StateMachine` ihre Stärke (die zeitliche Glättung) nicht voll ausspielen und die Katze muss warten bis der Zustandsautomat eine klare Entscheidung trifft.
3. **Farbe der Katze:** Wenn eine Katze ein sehr dunkles Fell besitzt, hat die Segmentierung Schwierigkeiten und verliert an Genauigkeit. Hier ist es essentiell Daten aus dem Praxisbetrieb zu sammeln, bei denen sich das Modell unsicher war, um genau diese Fälle besser trainieren zu können.

7.4 Lokale Datenverarbeitung

Ziel dieser Arbeit war es auch, dass das System ressourcenschonend konzipiert wird um sicherzustellen dass die Inferrenz auch auf einem Mikrocontroller lokal operieren kann. Daher viel die Entscheidung auf eine leichtgewichtige Architektur (YOLO26n) und optimierte Formate (TensorFlow Lite). Die dadurch verlorene Genauigkeit wird durch die erreichte Leistung gerechtfertigt (ca. 8 FPS).

Dieser Ansatz beweist, dass eine komplexe Aufgabe auf ressourcenbegrenzter Hardware durch eine intelligent gewählte Architektur gelöst werden kann, ohne dass die Robustheit zu stark beeinträchtigt wird. Dadurch wird eine Auslagerung von Berechnungen in externe Rechenzentren vermieden. Das spart Geld und macht die Lösung zuverlässiger, da keine Internetverbindung notwendig ist.

Abschließend lässt sich sagen, dass die Architektur ihre Aufgabe größtenteils erfüllt. Die verbleibenden Fehlalarme sind kein Architekturfehler, sondern zeigen die Grenzen des begrenzten Segmentierungsdatensatzes auf. Mithilfe des in Kapitel 5 integrierten Modells zum Speichern von Bildern, bei denen das Modell unsicher war, können genau diese Lücken im Datensatz ergänzt werden. So kann das Modell kontinuierlich verbessert werden.

Kapitel 8

Zusammenfassung und Ausblick

In der vorliegenden Bachelorarbeit wird ein robustes Zugangssystem für Katzen entwickelt, dass das eindringen von Beutetieren (z. B. Mäuse oder Vögel) in den Wohnraum verhindert. Durch die Kombination von mehrschitigen Lernen (eng. Deep Learning) und zeitlicher Aggregation wurde ein System geschaffen, das die einzelbildbasierte Objekterkennung bei Weitem übertrifft.

8.1 Zusammenfassung der Erkenntnisse

Die theoretischen Annahmen konnten durch die praktische Evaluierung erfolgreich validiert werden. Ein klassischer In diesem Anwendungsfall ist der monolithische Objektdetektor bei weitem ungenauer als die in der Arbeit vorgestellte Verarbeitungskette, die aus einem Objektdetektor, einer Bildsegmentierung und einer zeitlichen Aggregation besteht.

Zunächst detektiert ein YOLO-Detektor das Gesicht der Katze. Um zu verhindern, dass die eventuelle Beute im Maul der Katze okkludiert wird, wird der Ausschnitt des Gesichts um die doppelte Gesichtshöhe nach unten erweitert. Im zweiten Schritt analysiert ein YOLO-Segmentierungsmodell, ob sich Beute im Maul der Katze befindet. Um Fehlklassifikationen auszugleichen, muss eine Katze beweisen, dass sie sauber ist. Erst dann darf sie in den Wohnraum eintreten. Dazu wird die Katze über mehrere Sekunden hinweg mithilfe von ByteTrack verfolgt. Der gleitende Durchschnitt der letzten 30 Einzelbilder (was etwa zwei Sekunden entspricht) entscheidet dann, ob die Katze ins Haus darf oder nicht.

Die Evaluierung mit realen Videodaten zeigte den Erfolg des gewählten Architektur-Ansatzes. Das System konnte alle Testdaten erfolgreich und korrekt auswerten und so feststellen, ob die Katze sauber ist oder nicht. Der gewählte Schwellenwert von 0,8 spiegelt dabei die Priorität wider, dass Katzen im Falle von Systemunsicherheiten Zugang zum Haus haben. Dadurch wird sichergestellt, dass keine Katze zu Unrecht ausgesperrt wird. Das System erweist sich somit als sehr praxistauglich und läuft mit durchschnittlich 8 Bildern pro Sekunde ressourcenschonend auf lokalen Endgeräten.

Somit konnte die zentrale Forschungsfrage bejaht werden. Eine Kombination aus Objekterkennung, Objektverfolgung und zeitlicher Glättung ist deutlich robuster als herkömmliche Einzelbildauswertungen.

8.2 Ausblick und Weiterentwicklungsmöglichkeiten

Obwohl das System in seiner aktuellen Form bereits zuverlässig Beutetiere aussperrt, zeigt die Praxis, dass auch dieser Ansatz Schwächen und Limitierungen hat. Das eröffnet die Möglichkeit, konkrete Ansatzpunkte für zukünftige Verbesserungen zu identifizieren.

8.2.1 Kontinuierliches Sammeln schwieriger Negativbeispiele

Die vielversprechendste Methode, um die Systemleistung zu steigern, ist die Erweiterung des Datensatzes für Katzen mit Beute. Um dieses Ziel zu automatisieren, verfügt die Applikation über eine Funktionalität, die Bilder speichert, bei denen die Bildsegmentierung unsicher war. Dadurch können zukünftige Arbeiten diesen stetig wachsenden Datensatz nutzen, um das YOLO-Segmentierungsmodell iterativ nachzutrainieren. Sobald das Segmentierungsmodell aus diesen Fehlern lernen kann, wird die Fehlalarmrate des gesamten Systems erheblich sinken.

8.2.2 Verlust der Tiefeninformationen

Eine weitere Einschränkung ist der Verlust von 3D-Informationen im Bild. Dadurch kann es passieren, dass Objekte im Hintergrund oder sogar der Körper der Katze fälschlicherweise als „im Maul der Katze“ interpretiert werden. Eine mögliche Erweiterung bestünde darin, diese 3D-Informationen mithilfe von Stereo-Kameras oder Time-of-Flight-Sensoren zurückzugewinnen. Dadurch könnten Fehlalarme herausgefiltert werden, bei denen sich das vermeintliche Beutetier weit hinter dem Maul der Katze befindet.

8.2.3 Hardware-Beschleunigung

Derzeit wird das System ausschließlich auf der CPU ausgeführt. Ein Wechsel auf eine dedizierte Hardware (z. B. Google Coral) würde die Anzahl der pro Sekunde verarbeiteten Bilder deutlich erhöhen. Dadurch können wir in der gleichen Zeit mehr Daten der Katze sammeln, was die zeitliche Glättung verbessern würde. Das Flackern des Systems würde dadurch noch weiter eingeschränkt.

Abschließend lässt sich zusammenfassen, dass die in dieser Arbeit entwickelte Verarbeitungskette ein äußerst kompaktes und robustes Fundament bildet. Sie löst das Problem des unerwünschten Beuteeintrags durch Katzen zuverlässig und ermöglicht zugleich spannende Erweiterungen und Verbesserungen.

Quellenverzeichnis

Literatur

[0] Martín Abadi u. a. „Tensorflow: A system for large-scale machine learning“. In: *12th USENIX symposium on operating systems design and implementation (OSDI 16)*. 2016, S. 265–283 (siehe S. 6).

[0] Arm Limited. *Arm Architecture*. Verfügbar unter: https://www.arm.com/architecture. 2026 (siehe S. 6).

[0] Alex Bewley u. a. „Simple Online and Realtime Tracking“. In: *IEEE International Conference on Image Processing (ICIP)*. 2016, S. 3464–3468 (siehe S. 3).

[0] Daniel Bolya u. a. „YOLACT: Real-Time Instance Segmentation“. In: *Proceedings of the IEEE/CVF International Conference on Computer Vision (ICCV)*. 2019, S. 9157–9166 (siehe S. 3).

[0] Gary Bradski. „The OpenCV Library“. *Dr. Dobb's Journal of Software Tools* (2000) (siehe S. 6).

[0] Crawford. *Cat Dataset*. Verfügbar unter: https://www.kaggle.com/datasets/crawford/cat-dataset. 2024 (siehe S. 5, 8, 27).

[0] Mark Everingham u. a. „The PASCAL Visual Object Classes (VOC) Challenge“. *International Journal of Computer Vision* 88.2 (2010), S. 303–338 (siehe S. 16).

[0] Gemini Team u. a. „Gemini: a family of highly capable multimodal models“. *arXiv preprint arXiv:2312.11805* (2023) (siehe S. 8, 21).

[0] Ian Goodfellow, Yoshua Bengio und Aaron Courville. *Deep Learning*. MIT Press, 2016 (siehe S. 8).

[0] Wei Han u. a. *Seq-NMS for Video Object Detection*. arXiv:1602.08465. 2016. arXiv: 1602.08465 (siehe S. 3).

[0] David Harel. „Statecharts: A Visual Formalism for Complex Systems“. *Science of Computer Programming* 8.3 (1987), S. 231–274 (siehe S. 4).

[0] Kaiming He u. a. „Mask R-CNN“. In: *Proceedings of the IEEE International Conference on Computer Vision (ICCV)*. 2017, S. 2961–2969 (siehe S. 3).

[0] John E. Hopcroft, Rajeev Motwani und Jeffrey D. Ullman. *Introduction to Automata Theory, Languages, and Computation*. 3. Aufl. Pearson Addison-Wesley, 2006 (siehe S. 4).

[0] Licheng Jiao u. a. „New Generation Deep Learning for Video Object Detection: A Survey“. *IEEE Transactions on Neural Networks and Learning Systems* (2022) (siehe S. 1).

[0] Glenn Jocher u. a. *Ultralytics YOLO26: Unified Real-Time End-to-End Vision Models*. arXiv:2606.03748. 2026. arXiv: 2606.03748 (siehe S. 1–3).

[0] Rudolph Emil Kalman. „A new approach to linear filtering and prediction problems“. *Journal of basic Engineering* 82.1 (1960), S. 35–45 (siehe S. 3).

[0] Kai Kang u. a. „T-CNN: Tubelets With Convolutional Neural Networks for Object Detection From Videos“. *IEEE Transactions on Circuits and Systems for Video Technology* 28.10 (2018), S. 2896–2907 (siehe S. 3).

[0] Alex Krizhevsky, Ilya Sutskever und Geoffrey E. Hinton. „ImageNet Classification with Deep Convolutional Neural Networks“. In: *Advances in Neural Information Processing Systems (NeurIPS)*. Bd. 25. 2012 (siehe S. 2).

[0] Harold W. Kuhn. „The Hungarian Method for the Assignment Problem“. *Naval Research Logistics Quarterly* 2.1–2 (1955), S. 83–97 (siehe S. 3).

[0] Yann LeCun u. a. „Gradient-based learning applied to document recognition“. *Proceedings of the IEEE* 86.11 (1998), S. 2278–2324 (siehe S. 2, 3).

[0] Tsung-Yi Lin u. a. „Microsoft COCO: Common Objects in Context“. In: *Computer Vision – ECCV 2014*. Bd. 8693. Lecture Notes in Computer Science. Springer, 2014, S. 740–755 (siehe S. 16).

[0] Jonathan Long, Evan Shelhamer und Trevor Darrell. „Fully Convolutional Networks for Semantic Segmentation“. In: *Proceedings of the IEEE Conference on Computer Vision and Pattern Recognition (CVPR)*. 2015, S. 3431–3440 (siehe S. 3).

[0] Wenhan Luo u. a. „Multiple Object Tracking: A Literature Review“. *Artificial Intelligence* 293 (2021), S. 103448 (siehe S. 3).

[0] George H. Mealy. „A Method for Synthesizing Sequential Circuits“. *The Bell System Technical Journal* 34.5 (1955), S. 1045–1079 (siehe S. 4).

[0] Shervin Minaee u. a. „Image Segmentation Using Deep Learning: A Survey“. *IEEE Transactions on Pattern Analysis and Machine Intelligence* 44.7 (2022), S. 3523–3542 (siehe S. 3).

[0] Edward F. Moore. „Gedanken-Experiments on Sequential Machines“. In: *Automata Studies*. Hrsg. von Claude E. Shannon und John McCarthy. Bd. 34. Annals of Mathematics Studies. Princeton University Press, 1956, S. 129–153 (siehe S. 4).

[0] Joseph Redmon u. a. „You Only Look Once: Unified, Real-Time Object Detection“. In: *Proceedings of the IEEE Conference on Computer Vision and Pattern Recognition (CVPR)*. 2016, S. 779–788 (siehe S. 2).

[0] Shaoqing Ren u. a. „Faster R-CNN: Towards Real-Time Object Detection with Region Proposal Networks“. In: *Advances in Neural Information Processing Systems (NeurIPS)*. Bd. 28. 2015 (siehe S. 2).

[0] Ranjan Sapkota u. a. *YOLO26: Key Architectural Enhancements and Performance Benchmarking for Real-Time Object Detection*. arXiv:2509.25164. 2025. arXiv: 25 09.25164 (siehe S. 2).

[0] Sure Petcare. *Microchip Cat Flap*. Verfügbar unter: https://www.surepetcare.com/en-gb/pet-doors/microchip-cat-flap. 2026 (siehe S. 13).

[0] Richard Szeliski. *Computer Vision: Algorithms and Applications*. Springer Nature, 2022 (siehe S. 2).

[0] Juan Terven, Diana-Margarita Córdova-Esparza und Julio-Alejandro Romero-González. „A Comprehensive Review of YOLO Architectures in Computer Vision: From YOLOv1 to YOLOv8 and YOLO-NAS“. *Machine Learning and Knowledge Extraction* 5.4 (2023), S. 1680–1716 (siehe S. 2).

[0] Ultralytics. *Glossary: Confidence*. Verfügbar unter: https://www.ultralytics.com/de/glossary/confidence. 2026 (siehe S. 2, 17).

[0] Ultralytics. *YOLO Data Augmentation*. Verfügbar unter: https://docs.ultralytics.com/guides/yolo-data-augmentation. 2024 (siehe S. 8).

[0] Guido Van Rossum und Fred L Drake Jr. *Python tutorial*. Centrum voor Wiskunde en Informatica Amsterdam, The Netherlands, 1995 (siehe S. 6).

[0] Nicolai Wojke, Alex Bewley und Dietrich Paulus. „Simple Online and Realtime Tracking with a Deep Association Metric“. In: *IEEE International Conference on Image Processing (ICIP)*. 2017, S. 3645–3649 (siehe S. 3).

[0] Jason Yosinski u. a. „How Transferable Are Features in Deep Neural Networks?“ In: *Advances in Neural Information Processing Systems (NeurIPS)*. Bd. 27. 2014 (siehe S. 8).

[0] Yifu Zhang u. a. „ByteTrack: Multi-Object Tracking by Associating Every Detection Box“. In: *Computer Vision – ECCV 2022*. Bd. 13682. Lecture Notes in Computer Science. Springer, 2022, S. 1–21 (siehe S. 1, 3).

[0] Zhengxia Zou u. a. „Object Detection in 20 Years: A Survey“. *Proceedings of the IEEE* 111.3 (2023), S. 257–276 (siehe S. 2).